DS	**Session**	Tree 2	**Question Information**	Level 1 Challenge 71

Question description

Given two rooted trees, your task is to find out if they are *isomorphic*, i.e., it is possible to draw them so that they look the same.

Constraints

- $1 \le t \le 1000$
- $2 \le n \le 10^5$
- the sum of all values of n is at most 10^5

Input

The first input line has an integer t: the number of tests. Then, there are t tests described as follows:
The first line has an integer n: the number of nodes in both trees. The nodes are numbered 1,2,...,n, and node 1 is the root.
Then, there are n−1 lines describing the edges of the first tree, and finally n−1 lines describing the edges of the second tree.

Output

For each test, print "YES", if the trees are isomorphic, and "NO" otherwise.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
3
1 2
2 3
1 2
1 3
3
1 2
2 3
1 3
3 2
```

Test Case 2

INPUT (STDIN)

```
2
3
1 2
2 3
1 3
3 3
3
1 2
2 3
1 3
3 3
```

EXPECTED OUTPUT

```
NO
YES
```

EXPECTED OUTPUT

```
NO
NO
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(t--)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
2
```

Test Case 2

TOKEN COUNT

```
480
```

Test Case 3

NLOC

```
65
```

Code

isomorphic_trees.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 100005
#define MOD 1000000007ULL
#define BASE 1000003ULL

typedef struct Edge {
    int to;
    struct Edge* next;
} Edge;

Edge* head1[MAXN];
Edge* head2[MAXN];
Edge edge_pool[4 * MAXN];
int pool_ptr = 0;

void add_edge(Edge** head, int u, int v) {
    edge_pool[pool_ptr].to = v;
    edge_pool[pool_ptr].next = head[u];
    head[u] = &edge_pool[pool_ptr++];

    edge_pool[pool_ptr].to = u;
    edge_pool[pool_ptr].next = head[v];
    head[v] = &edge_pool[pool_ptr++];
}

int cmp_ull(const void* a, const void* b) {
    unsigned long long x = *(const unsigned long long*)a;
    unsigned long long y = *(const unsigned long long*)b;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

unsigned long long get_hash(Edge** head, int u, int p) {
    unsigned long long child_hashes[100];
    int deg = 0;

    for (Edge* e = head[u]; e; e = e->next) {
        if (e->to != p) {
            child_hashes[deg++] = get_hash(head, e->to, u);
        }
    }

    qsort(child_hashes, deg, sizeof(unsigned long long), cmp_ull);
    unsigned long long h = 1999ULL;
    for (int i = 0; i < deg; i++) {
        h = (h * BASE + child_hashes[i]) % MOD;
    }
    return (h ^ (h >> 16)) * 2654435761ULL;
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;

    while(t--) {
        int n;
        if (scanf("%d", &n) != 1) break;
        pool_ptr = 0;
        for (int i = 1; i <= n; i++) {
            head1[i] = NULL;
            head2[i] = NULL;
        }

        for (int i = 0; i < n - 1; i++) {
            int u, v;
            scanf("%d %d", &u, &v);
            add_edge(head1, u, v);
        }
        for (int i = 0; i < n - 1; i++) {
            int u, v;
            scanf("%d %d", &u, &v);
            add_edge(head2, u, v);
        }

        unsigned long long h1 = get_hash(head1, 1, 0);
        unsigned long long h2 = get_hash(head2, 1, 0);

        if (h1 == h2) {
            printf("YES\n");
        } else {
            printf("NO\n");
        }
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc isomorphic_trees.c -o isomorphic_trees
PS C:\C Programming> .\isomorphic_trees
1
3
1 2
1 3
1 2
2 3
NO
PS C:\C Programming>